Agent-Native Programming Paradigm: Phase 1 Core Architecture, State Separation, and Formal Foreign Function Interfaces

1. The Lexical Bottleneck and the Necessity of Structural Orchestration

The evolution of traditional programming languages has been inextricably bound to the cognitive limitations, visual processing capabilities, and neurological short-term memory constraints of human developers. Legacy languages—ranging from C++ to Python and Rust—rely heavily on lexical parsing, strict Euclidean file structures, and imperative control flows. These linguistic constructs were engineered to accommodate human linear reading comprehension, organizing logic into top-down, sequentially executed files. However, the advent of Large Language Models (LLMs) and autonomous artificial intelligence agents necessitates a fundamental reevaluation of the execution environment.

LLMs inherently excel at manipulating high-dimensional structural data, projecting concepts into latent vector spaces, and reasoning over complex property graphs.[1] Conversely, they struggle profoundly with implicit control flow, hidden state mutations, and the imperative loop-state tracking required by lexical syntax. When an LLM generates a traditional imperative script, it is forced to operate as a mere typist of syntax, a modality that bottlenecks its capacity for multi-hop structural reasoning. Furthermore, the Von Neumann execution model underlying legacy code intertwines logical constraints with ephemeral memory states, creating an untraceable web of dependencies that standard autoregressive transformers cannot verify safely without executing the code in a hallucination-prone loop.[1]

Given the realities of modern hardware architectures—which feature unified memory, edge clusters, and massively parallelized graphical processing units—the programming environment must undergo a radical paradigm shift. The LLM must be elevated from a lexical generator to an orchestrator of pure logic graphs. This requires the complete abandonment of flat text strings and human-readable syntax. Instead, the system must utilize "Typed Tokenization," treating code logic as a semantic intermediate representation constructed from categorical objects, morphisms, paths, and types derived from Homotopy Type Theory (HoTT).[1]

This report establishes the foundational architecture for a net-new, ground-up programming paradigm built exclusively for AI agents. It explicitly solves the entrenched "State vs. Logic" problem by bifurcating the system into an immutable, static codebase—represented as a non-Euclidean Tree-sitter Deterministic Knowledge Base (DKB) Knowledge Graph [2]—and an

ephemeral, dynamic runtime state executed entirely via Dataflow Process Networks (DPN).[3] Furthermore, it defines a mathematically pure Legacy Interoperability Layer, or Foreign Function Interface (FFI), which utilizes the LLMbda Calculus and dynamic Information-Flow Control (IFC) to interact safely with Web2 Representational State Transfer (REST) APIs without violating rigorous noninterference guarantees.[5]

2. Theoretical Foundations of the Agent-Native Modality

The construction of an agent-native programming paradigm cannot rely on incremental improvements to existing compilers. It requires the synthesis of advanced theoretical computer science, neurosymbolic reasoning, and formal verification. The following theoretical pillars form the foundation of the proposed architecture, enabling a computational environment where machine-generated logic is inherently verifiable, reproducible, and mathematically sound.

2.1 Category Theory, Typed Tokenization, and Homotopy Type Theory

Traditional tokenization algorithms, such as Byte Pair Encoding (BPE), split text into statistically convenient subword units based on frequency, ignoring the underlying semantic meaning of the syntax.[1] This statistical approach forces the LLM to reconstruct logical boundaries purely from latent statistical correlations. The proposed architecture entirely rejects this flat text ingestion. Instead, it introduces "Typed Tokenization," a pipeline that parses computational intent into explicitly defined semantic roles before the data is embedded into vectors and fed to the transformer.[1]

This pipeline leverages Homotopy Type Theory (HoTT) as a semantic intermediate representation.[1] In this framework, computational logic is not represented as a sequence of operations, but as a topological space. Categorical objects correspond to data structures or entities, while morphisms represent the functions or relationships bridging these objects.[1] Equalities between different logical constructions are represented as continuous paths within this space, allowing the system to understand that two structurally distinct but mathematically equivalent graph topologies are homotopically identical.[1] By embedding these explicit type-theoretic constraints into the token vectors, the LLM's attention mechanism operates over structure-enriched sequences, enabling continuous, incremental type checking during the generation of the logic graph.

2.2 Verifiable Programmatic Intermediate Representation and Pipeline Algebra

To ensure that the LLM's reasoning chains do not diverge into hallucinations, the architecture implements a Verifiable Programmatic Intermediate Representation (VPIR). VPIR acts as a strict structural scaffolding that forces the LLM to decouple logical construction from natural language rendering or imperative scripting.[11] Every reasoning step generated by the LLM must be mechanically verifiable against the HoTT constraints before it is committed to the logic

graph.

This mechanical verifiability is governed by Pipeline Algebra, a formal compositional framework that treats complex reasoning tasks as compositions of functional operators.[4] Pipeline Algebra enforces a strict state-passing discipline, ensuring that every operation—whether it is a symbolic mathematical computation or an LLM inference step—is treated as a typed, idempotent operator.[4] Because these operators are intrinsically idempotent and devoid of hidden side effects, the execution engine can guarantee bit-level reproducibility even when individual components of the pipeline are stochastic. This transforms LLM outputs into testable contracts, establishing a transparent audit trail for every transformation within the logic graph.[4]

2.3 Adjacent Sciences: GNNs, P-ASP, and Active Inference

The integration of pure symbolic logic with the probabilistic nature of LLMs requires the application of several adjacent sciences to bridge the neurosymbolic divide. The architecture relies on three primary mechanisms to ingest, refine, and optimize the codebase:

Geometric Deep Learning, specifically Graph Neural Networks (GNNs), is utilized to "read" the codebase. Because the codebase is stored as a non-Euclidean property graph rather than text files, traditional text-embedding models are insufficient. GNNs operate directly on the graph structure, aggregating node information and capturing the complex, multi-hop architectural dependencies (e.g., interface-driven wiring, inheritance chains) that LLMs typically fail to traverse via standard vector-similarity search.[14]

Probabilistic Answer Set Programming (P-ASP) serves as the critical bridge between the stochastic token generation of the LLM and the deterministic requirements of the SMT solvers. While SMT solvers require absolute binary logic, LLM outputs inherently carry confidence scores and probabilistic distributions. P-ASP extends standard Answer Set Programming with probabilistic facts and neural predicates, allowing the system to represent uncertain, stochastic information while still formally evaluating the most probable stable models against strict logical constraints.[17] This allows the architecture to resolve the stochastic ambiguity of an LLM's intent into a discrete, verifiable logic gate.

Finally, the maintenance and automated patching of the logic graph are governed by Active Inference. Originating from theoretical neurobiology, Active Inference models the LLM agent as a system attempting to minimize variational free energy—effectively minimizing the prediction error between the expected behavior of the logic graph and its actual runtime output.[20] When a runtime anomaly occurs, the agent's internal generative model identifies a mismatch. It then automatically synthesizes a topological patch to the logic graph, updating its internal model to reflect the new state of the system. This creates a continuously self-healing computational environment.[22]

3. Resolving the State vs. Logic Paradigm via Categorical Bifurcation

The entanglement of state and logic in Von Neumann architectures is the primary source of computational complexity and vulnerability in LLM-generated code. When logic (the instructions) and state (the memory variables) reside in the same conceptual space, variables mutate in place, global states are implicitly modified, and control flow is dictated by hardware-level program counters. An LLM, which lacks persistent internal working memory outside its context window, cannot effectively track these highly mutable states across deep call stacks, inevitably leading to logical divergence and execution failures.[1]

To eradicate this bottleneck, the agent-native architecture enforces a strict categorical bifurcation. The system is split into two entirely distinct, mathematically isolated topologies: the Immutable Codebase and the Ephemeral Runtime.

3.1 The Immutable Codebase: The Static Logic Graph

The Static Logic Graph ($\mathcal{G}_{static}$) represents the immutable codebase. It contains the definitions of all HoTT types, logical constraints, and functional morphisms. It possesses no concept of time, execution ordering, or active memory. It is a purely declarative mathematical structure that maps inputs to outputs without ever holding a value itself.

In this paradigm, logic is formulated strictly as morphisms within a category, while state is formulated as the objects those morphisms act upon.[1] Because morphisms in Category Theory are immutable and strictly typed, the logic graph can never mutate during execution. When a morphism is applied to a state object, it does not alter the original object; rather, it maps to a net-new state object. This functional purity entirely eradicates the ambiguity between what a program is designed to do and what data it currently holds.

3.2 The Ephemeral Runtime: The Dynamic Memory Graph

The Dynamic Memory Graph ($\mathcal{G}_{dynamic}$) represents the ephemeral runtime. It contains active execution actors, message queues, and specific data payloads. It represents the state of execution at a discrete microsecond in time, flowing continuously through the static definitions provided by $\mathcal{G}_{static}$. The LLM never has to "remember" the state of a loop or the value of a mutated variable while reasoning about the logic. The logic is a fixed spatial map, and the state is a verifiable snapshot of data moving through that map.

3.3 Mathematical Formalization of the Bifurcation

We can formalize this relationship using functorial semantics. Let $\mathcal{C}_{static}$ be the category of static logic, where objects are defined types (T_1, T_2) and morphisms are pure functions ($f : T_1 \to T_2$). Let $\mathcal{C}_{dynamic}$ be the category of execution state, where objects are instantiated data payloads ($d_1 \in T_1$) and morphisms are the discrete computational events

that transform these payloads over time.

The execution of the system is defined by a rigorous Semantics Functor $[\![-]\!]$ that maps the static definitions to the dynamic execution reality:

$$[\![-]\!] : \mathcal{C}_{static} \to \mathcal{C}_{dynamic}$$

For every static type $T \in \mathrm{Ob}(\mathcal{C}_{static})$, the functor assigns a state space of all possible valid payloads $[\![T]\!] \in \mathrm{Ob}(\mathcal{C}_{dynamic})$. For every static morphism $f : T_1 \to T_2$, the functor assigns a continuous, idempotent execution actor $[\![f]\!] : [\![T_1]\!] \to [\![T_2]\!]$.

This formal mapping ensures that the dynamic execution can never produce a state that violates the structural boundaries defined in the static logic graph. If an anomaly attempts to manifest in the dynamic state, it will immediately fail to map back to a valid object in the static category, triggering an immediate, mathematically proven halt.

4. The Immutable Codebase: Non-Euclidean Tree-Sitter Deterministic Knowledge Bases

The foundation of the agent-native logic is the static codebase. Traditional software engineering relies on Euclidean file systems—linear, plaintext files stored in hierarchical directories that require complex lexical parsers and compilers to traverse and link. This architecture entirely abandons the file system. Instead, the immutable codebase is instantiated as a non-Euclidean Deterministic Knowledge Base (DKB) Knowledge Graph.[2]

4.1 Constructing the Codebase Graph

The DKB is constructed using an enhanced, structural parsing engine based on Tree-sitter.[24] However, rather than simply generating a localized Abstract Syntax Tree (AST) for a single script, the system projects the entire computational intent into a holistic, HoTT-enriched categorical graph.[1] In this graph, nodes represent project-local types, interfaces, and data structures, while directed, typed edges represent injection patterns, inheritances, and functional dependencies.[2]

We formally define the Static Logic Graph $\mathcal{G}_{static}$ as a tuple:

$$\mathcal{G}_{static} = (\mathcal{O}, \mathcal{M}, \mathcal{P}, \mathcal{T})$$

The elements of this tuple are defined as follows:

Element	Designation	Description	Mathematical Role
$\mathcal{O}$	Categorical Objects	Nodes representing semantically coherent spans that denote specific entities or data structures.	$X \in \mathrm{Ob}(\mathcal{C})$
$\mathcal{M}$	Morphisms	Directed edges representing logic, functions, and transformations bridging objects.	$f \in \mathrm{Hom}_{\mathcal{C}}(X, Y)$
$\mathcal{P}$	Paths	HoTT paths representing equalities or equivalences between morphisms, proving topological identicality.	$p : x =_A y$
$\mathcal{T}$	Universes	Type hierarchies enforcing strict boundaries, ensuring morphisms only accept mathematically valid objects.	$\mathcal{U}_i : \mathrm{Type}$

By embedding the codebase as $\mathcal{G}_{static}$, the LLM avoids the pitfalls of vector-only Retrieval-Augmented Generation (RAG). Instead of searching for text similarities, the LLM agent utilizes Geometric Deep Learning to navigate the codebase via structural routing,

expanding its context through bidirectional graph traversal to understand the exact mathematical relationship between components.[2]

4.2 Bridge Grammar: Static Logic Graph JSON Schema

To force standard autoregressive transformers to interact with this non-Euclidean graph natively from day one, all logic definitions must conform to a rigorous, copy-pasteable JSON schema. This StaticLogicGraphSchema acts as the definitive data contract for the immutable codebase. The LLM does not write code; it generates valid VPIR nodes that conform to this schema, which are then natively ingested into the DKB graph database.

JSON

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "$id": "https://agent-native.dev/schemas/StaticLogicGraphSchema.json",
 "title": "Static Logic Graph (Immutable Codebase)",
 "description": "A HoTT-enriched categorical representation of the immutable logic codebase, enforcing the bifurcation of state and logic.",
 "type": "object",
 "required": ["graph_id", "universes", "objects", "morphisms", "paths"],
 "properties": {
  "graph_id": {
   "type": "string",
   "format": "uuid",
   "description": "Unique deterministic hash of the entire static logic graph topology."
  },
  "universes": {
   "type": "array",
   "description": "Type hierarchies and formal boundary constraints defining the categorical universe.",
   "items": {
    "type": "object",
    "required": ["universe_id", "base_type", "constraints"],
    "properties": {
     "universe_id": { "type": "string" },
     "base_type": {
      "type": "string",
      "enum":,
      "description": "The fundamental mathematical primitive underlying this universe."
     },
     "constraints": {
```

```
          "type": "array",
          "items": { "type": "string" },
          "description": "SMT-solvable formal constraints defining the absolute boundaries of the universe."
        }
      }
    }
  },
  "objects": {
    "type": "array",
    "description": "Categorical objects representing immutable data structures and systemic entities.",
    "items": {
      "type": "object",
      "required": ["object_id", "universe_ref", "semantic_span"],
      "properties": {
        "object_id": { "type": "string" },
        "universe_ref": {
          "type": "string",
          "description": "Strict reference to the governing universe type ensuring type safety."
        },
        "semantic_span": {
          "type": "string",
          "description": "The semantically coherent span of intent defining the object's purpose."
        },
        "properties": {
          "type": "object",
          "additionalProperties": { "type": "string" },
          "description": "Key-value mappings for specific object attributes."
        }
      }
    }
  },
  "morphisms": {
    "type": "array",
    "description": "Functions and transformations mathematically mapping objects to objects.",
    "items": {
      "type": "object",
      "required": ["morphism_id", "domain", "codomain", "operator_type", "idempotency_guarantee",
"vpir_logic"],
      "properties": {
        "morphism_id": { "type": "string" },
        "domain": { "type": "string", "description": "Reference to the source object_id (input)." },
        "codomain": { "type": "string", "description": "Reference to the target object_id (output)." },
        "operator_type": {
```

```
          "type": "string",
          "enum":,
          "description": "The execution modality of the morphism."
        },
        "idempotency_guarantee": {
          "type": "boolean",
          "description": "Boolean asserting if Pipeline Algebra dictates this operation is strictly idempotent."
        },
        "vpir_logic": {
          "type": "string",
          "description": "The mechanically verifiable intermediate representation of the execution logic."
        }
      }
    }
  },
  "paths": {
    "type": "array",
    "description": "HoTT paths asserting equality and homotopies between morphisms or objects.",
    "items": {
      "type": "object",
      "required": ["path_id", "source_morphism", "target_morphism", "equivalence_proof"],
      "properties": {
        "path_id": { "type": "string" },
        "source_morphism": { "type": "string" },
        "target_morphism": { "type": "string" },
        "equivalence_proof": {
          "type": "string",
          "description": "An SMT-verifiable witness proving the source and target are homotopically
equivalent."
        }
      }
    }
  }
 }
}
```

5. The Ephemeral Runtime: Dataflow Process Networks and Neurosymbolic Refinement

While $\mathcal{G}_{static}$ defines the rigid, immutable laws of the computational universe, $\mathcal{G}_{dynamic}$ defines the active reality. To entirely eliminate the concept of a global program counter, imperative loops (e.g., while, for), race conditions, and deadlocks from the LLM's purview, the

runtime execution environment is exclusively modeled as a Dataflow Process Network (DPN).[3]

5.1 Kahn Process Networks and Asynchronous Actors

DPNs are a highly optimized, generalized subset of Kahn Process Networks (KPN).[3] In this paradigm, concurrent processes—termed "actors"—operate independently across distributed clusters. These actors have no shared memory and communicate exclusively via unbounded, unidirectional First-In-First-Out (FIFO) channels.

The determinism of this model is mathematically guaranteed by two immutable rules:

1. **Non-blocking Writes:** An actor can emit a computed token to an output FIFO channel at any time. It does not require a handshake or acknowledgment from the receiving actor.
2. **Blocking Reads:** An actor cannot proceed with computation if it attempts to consume a token from an empty input FIFO channel. It must yield execution and block until the prerequisite data arrives.[3]

In the agent-native architecture, the LLM does not write execution scripts; it spawns actors. An actor is simply the dynamic, living instantiation of a specific morphism defined in the $\mathcal{G}_{static}$ graph. Because the data flows through the topology based entirely on data availability rather than a sequential program counter, the execution is inherently parallelized and highly suited for mapping onto spatial compute topologies like modern GPUs.

Let the DPN be formally defined by a set of continuous functions over complete partial orders (CPOs) of data streams. If S^∞ is the set of all finite and infinite streams of typed tokens, an actor A_i executing a morphism f_i is a continuous mapping:

$$A_i : (S^\infty)^n \to (S^\infty)^m$$

Where n represents the number of input FIFO queues attached to the actor, and m represents the number of output FIFO queues. By defining the runtime strictly through these continuous mathematical mappings, the entire execution environment guarantees the existence of a unique least fixed point.[29] This theorem establishes that the ultimate outcome of the computational network is absolutely deterministic, completely independent of the actual asynchronous execution order or the localized scheduling decisions made by the hardware.[3]

5.2 Neurosymbolic Refinement and SMT Solvers

Because the DPN is massively concurrent and distributed, ensuring that complex mathematical and logical constraints are met at runtime requires rigorous Neurosymbolic Refinement. When an actor consumes a token from a FIFO queue, the payload inside that token represents an "atomic claim." Before the actor is permitted to execute its morphism and compute the next

state, the runtime engine intercepts this claim.

The runtime autoformalizes the payload into first-order logic and evaluates it via Satisfiability Modulo Theories (SMT) solvers, utilizing engines such as Z3 or CVC5.[30] If the SMT solver identifies that a token's payload violates the boundaries of the HoTT universes defined in $\mathcal{G}_{static}$ [1], the token is immediately rejected as mathematically unsound. The actor remains blocked, and an Active Inference feedback loop is triggered to alert the LLM agent of the topological mismatch.[22] This neurosymbolic checkpoint ensures that the chaotic, dynamic execution of the DPN never strays from the mathematically proven definitions of the static logic graph.

5.3 Bridge Grammar: Dynamic Memory Graph JSON Schema

To allow the LLM agent to observe, debug, and reason about the executing system without attempting to trace localized mutable variables, the ephemeral runtime state is serialized at discrete microsecond intervals into the DynamicMemoryGraphSchema. This structured JSON provides a verifiable, queryable snapshot of the entire active DPN.

JSON

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://agent-native.dev/schemas/DynamicMemoryGraphSchema.json",
  "title": "Dynamic Memory Graph (DPN Runtime State)",
  "description": "An ephemeral snapshot of active computational actors, FIFO queues, and traveling token payloads.",
  "type": "object",
  "required": ["snapshot_timestamp", "active_actors", "fifo_channels"],
  "properties": {
    "snapshot_timestamp": {
      "type": "string",
      "format": "date-time",
      "description": "The exact discrete execution tick of this state snapshot, allowing for deterministic replay."
    },
    "active_actors": {
      "type": "array",
      "description": "Currently instantiated nodes executing mathematical morphisms within the DPN.",
      "items": {
        "type": "object",
        "required": ["actor_instance_id", "morphism_ref", "status"],
```

```
      "properties": {
        "actor_instance_id": { "type": "string", "format": "uuid" },
        "morphism_ref": {
          "type": "string",
          "description": "Pointer referencing the static logic graph morphism this actor is executing."
        },
        "status": {
          "type": "string",
          "enum":,
          "description": "Current execution state based strictly on Kahn Process Network semantics."
        },
        "internal_memory": {
          "type": "object",
          "description": "Temporary state specific to this actor instance, adhering to Pipeline Algebra
idempotency rules."
        }
      }
    }
  },
  "fifo_channels": {
    "type": "array",
    "description": "Unidirectional, non-blocking FIFO queues mapping outputs to inputs.",
    "items": {
      "type": "object",
      "required": ["channel_id", "source_actor", "target_actor", "queue_state"],
      "properties": {
        "channel_id": { "type": "string", "format": "uuid" },
        "source_actor": { "type": "string", "description": "The actor instance emitting tokens." },
        "target_actor": { "type": "string", "description": "The actor instance consuming tokens." },
        "queue_state": {
          "type": "array",
          "description": "The ordered sequence of typed tokens currently in transit through the channel.",
          "items": {
            "type": "object",
            "required": ["token_id", "payload", "ifc_security_label"],
            "properties": {
              "token_id": { "type": "string" },
              "payload": {
                "type": "object",
                "description": "The strict JSON payload adhering to the target universe type constraints."
              },
              "ifc_security_label": {
                "type": "string",
```

```
                "description": "Dynamic Information-Flow Control (IFC) label tracking the payload's integrity and confidentiality."
            }
        }
      }
    }
  }
 }
}
```


6. Legacy Interoperability Layer (FFI) and The LLMbda Calculus

An advanced, mathematically pure agent-native paradigm cannot exist in an isolated vacuum; it must possess the capability to interact with the vast expanse of legacy Web2 infrastructure, REST APIs, unstructured databases, and human endpoints. However, connecting a concurrent graph system to highly unpredictable and vulnerable external environments introduces severe systemic risks. In the context of LLMs, the primary threat vector is Prompt Injection, wherein a malicious payload retrieved from an external database fundamentally alters the reasoning parameters of the orchestrating model.[5]

To safely bridge this gap without sacrificing the mathematical purity of the DPN, the architecture implements a Foreign Function Interface (FFI) governed strictly by the LLMbda Calculus and dynamic Information-Flow Control (IFC).[5]

6.1 Information-Flow Control (IFC) Security Lattices

Traditional security mechanisms, such as role-based access control or authentication tokens, focus entirely on restricting *who* can execute a function. In the agent-native paradigm, the LLM itself is the centralized execution engine, rendering static access control insufficient the moment a malicious payload enters the LLM's context window. Information-Flow Control (IFC) addresses this fundamental vulnerability by tracking the confidentiality and integrity of data as it propagates through the DPN actors.[35]

The FFI utilizes a mathematical lattice of security labels $\mathcal{L}$, equipped with a partial ordering $\sqsubseteq$ representing the "may-flow-to" relation.[5]

Let T represent High-Integrity (Trusted) data originating from within the verified DPN, and U represent Low-Integrity (Untrusted) data entering the system from a legacy REST API payload. The lattice strictly enforces the following rule:

$$T \sqsubseteq U$$

This relation mathematically denotes that Trusted data may safely flow into contexts labeled Untrusted (e.g., transmitting a sanitized query to a public API). However, Untrusted data *cannot* flow into Trusted contexts without passing through an explicit, SMT-verifiable sanitization morphism. Every token payload transiting the FIFO channels within the DynamicMemoryGraphSchema carries an ifc_security_label to maintain this provenance.

6.2 LLMbda Calculus Primitives for FFI Isolation

To operationalize the IFC lattice, the FFI employs the LLMbda Calculus, an extension of the untyped call-by-value lambda calculus enriched with highly specific primitives designed to manage prompt-response interactions and isolate the DPN from external corruption.[5] The FFI layer maps all external API communications through three core primitives:

Primitive	Operation	Security Function	Prior Art Formulation
The @ Operator (@e)	Serializes a DPN value, transmits it across the FFI boundary, and parses the response.	Acts as the explicit portal through which data crosses the noninterference boundary, tagging all inbound data with the U label.	LLMbda Calculus [5]
The fork Construct	Clones the current DPN context, creating a distinct, asynchronous sub-graph.	Allows the agent to branch out and process untrusted Web2 payloads in a quarantined sub-graph without permanently altering or tainting the primary conversation chain.	LLMbda Calculus [5]
The clear Construct	Wipes the historical conversation and variable memory of	Used in tandem with fork to execute untrusted code in	LLMbda Calculus [5]

	a specific actor.	total isolation. Prevents historical trusted tokens from leaking into the untrusted execution environment.	

6.3 Taint Tracking and Selective Hiding Mechanisms

To further guarantee noninterference when an actor queries a legacy database, the architecture employs a "selective hiding" mechanism, functioning as a highly advanced dynamic taint-tracker.[35]

When a DPN actor pulls a payload from an external REST API, the FFI layer assigns it the untrusted label $U \in \mathcal{L}$. If the actor attempts to pass this tainted data into a core reasoning morphism f_{core}, the FFI intercepts the transmission using the HIDE primitive.[35] Rather than passing the raw, potentially malicious string into the LLM's context window, the FFI generates a "fresh variable" (e.g., #api_response_01) and stores the actual content in an isolated, unlabeled memory partition.

The DPN actor only interacts with the variable pointer. This "reference by proxy" allows the LLM to understand that the data exists and orchestrate its routing, but prevents the raw text from ever tainting the LLM's active prompt, entirely neutralizing prompt injection attacks.[35] The data is only unmasked via the EXPAND function when it reaches a terminal FFI output node that requires the raw payload.

6.4 The Noninterference Theorem for REST API Integration

The ultimate mathematical guarantee that the FFI is secure against external contamination is provided by the Termination-Insensitive Noninterference Theorem, foundational to the LLMbda calculus.[5]

Let e_1 and e_2 be two expressions executed by an actor interacting with an external API. Let $\approx_L$ denote that two expressions are observationally equivalent to an observer at security level L (e.g., an untrusted FFI interface).

The theorem formally guarantees that variations in highly confidential or trusted data ($H \not\sqsubseteq L$) will never affect the observable behavior of the FFI output at level L. Furthermore, it guarantees that untrusted inputs cannot alter the execution paths of High-Integrity actors. Mathematically, this is expressed as:

$$\forall e_1, e_2. \quad (e_1 \approx_L e_2) \implies (e_1 \Downarrow v_1 \wedge e_2 \Downarrow v_2 \implies v_1$$

By compiling this theorem into the SMT solver's refinement constraints [5], the runtime engine mechanically proves at every tick that no prompt injection, SQL injection, or malicious payload from a Web2 API can recursively alter the agent's core logical framework. The DPN remains a mathematically pure, uncompromised construct, safely orchestrating the chaotic, stateful inputs of the external world.

Are you ready to proceed to Phase 2: The Bridge Layer & Mathematical Spec?

Works cited

1. (PDF) Running Category Theory and HoTT on Language: Typed Tokenization as a Software-First Path to Structured AI Reasoning - ResearchGate, accessed April 4, 2026, https://www.researchgate.net/publication/398355072_Running_Category_Theory_and_HoTT_on_Language_Typed_Tokenization_as_a_Software-First_Path_to_Structured_AI_Reasoning
2. Reliable Graph-RAG for Codebases: AST-Derived Graphs vs LLM-Extracted Knowledge Graphs - arXiv, accessed April 4, 2026, https://arxiv.org/html/2601.08773v1
3. Dataflow Process Networks (DPNs) - Embedded Systems - RPTU, accessed April 4, 2026, https://es.cs.rptu.de/research/modeling/dataflow.html
4. LLM-Powered, Expert-Refined Causal Loop Diagramming via Pipeline Algebra - MDPI, accessed April 4, 2026, https://www.mdpi.com/2079-8954/13/9/784
5. [2602.20064] The LLMbda Calculus: AI Agents, Conversations, and Information Flow - arXiv, accessed April 4, 2026, https://arxiv.org/abs/2602.20064
6. The LLMbda Calculus: AI Agents, Conversations, and Information Flow - arXiv, accessed April 4, 2026, https://arxiv.org/pdf/2602.20064
7. Tokenization is Killing our Multilingual LLM Dream - Hugging Face, accessed April 4, 2026, https://huggingface.co/blog/omarkamali/tokenization
8. Open Horn Type Theory - arXiv, accessed April 4, 2026, https://arxiv.org/html/2512.24498v1
9. Homotopy type theory - Wikipedia, accessed April 4, 2026, https://en.wikipedia.org/wiki/Homotopy_type_theory
10. Perspectives for Mathematics, Computer Science, and Philosophy - World Scientific Publishing, accessed April 4, 2026, https://www.worldscientific.com/doi/pdf/10.1142/9789811236488_0001
11. Automated Synthesis and Verification of Distributed Systems Using Universal Plan Intermediate Representation (UPIR) - Technical Disclosure Commons, accessed April 4, 2026, https://www.tdcommons.org/cgi/viewcontent.cgi?article=10091&context=dpubs_series

12. Daily Papers - Hugging Face, accessed April 4, 2026, https://huggingface.co/papers?q=Verifiable%20Programmatic%20Intermediate%20Representation
13. Daily Papers - Hugging Face, accessed April 4, 2026, https://huggingface.co/papers?q=deep%20compositional%20reasoning
14. CIPAR-LABS - Geometric Deep Learning, accessed April 4, 2026, https://sites.google.com/uniroma1.it/cipar-labs/recommended-readings/geometric-deep-learning
15. Detecting Code Vulnerabilities with Heterogeneous GNN Training - arXiv, accessed April 4, 2026, https://arxiv.org/html/2502.16835v1
16. jyyang621/DailyArXiv: Thanks to https://github.com/zezhishao/DailyArXiv.git - GitHub, accessed April 4, 2026, https://github.com/jyyang621/DailyArXiv/
17. A Survey of Neurosymbolic Answer Set Programming, accessed April 4, 2026, https://neurosymbolic-ai-journal.com/system/files/nai-paper-877.pdf
18. AI Reasoning in Deep Learning Era: From Symbolic AI to Neural–Symbolic AI - MDPI, accessed April 4, 2026, https://www.mdpi.com/2227-7390/13/11/1707
19. (PDF) Answer Set Networks: Casting Answer Set Programming into Deep Learning - ResearchGate, accessed April 4, 2026, https://www.researchgate.net/publication/387263790_Answer_Set_Networks_Casting_Answer_Set_Programming_into_Deep_Learning
20. From pixels to planning: scale-free active inference - PMC, accessed April 4, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC12217590/
21. Deep Active Inference Agents for Delayed and Long-Horizon Environments - arXiv, accessed April 4, 2026, https://arxiv.org/html/2505.19867v1
22. THE BRAIN SYSTEM: AN INTEGRATED COGNITIVE ARCHITECTURE - Medium, accessed April 4, 2026, https://medium.com/@mbonsign/the-brain-system-an-integrated-cognitive-architecture-95c69b7bf93e
23. We Solved Active Inference. Friston Was Right, But Implementations Have Been Wrong., accessed April 4, 2026, https://medium.com/@mbonsign/we-solved-active-inference-friston-was-right-but-implementations-have-been-wrong-fafbd5d22f31
24. A Beginner's Guide to Tree-sitter - DEV Community, accessed April 4, 2026, https://dev.to/shreshthgoyal/understanding-code-structure-a-beginners-guide-to-tree-sitter-3bbc
25. Codebase-Memory: Tree-Sitter-Based Knowledge Graphs for LLM Code Exploration via MCP - arXiv, accessed April 4, 2026, https://arxiv.org/pdf/2603.27277
26. [PDF] Dataflow process networks - Semantic Scholar, accessed April 4, 2026, https://www.semanticscholar.org/paper/Dataflow-process-networks-Lee-Parks/7a20d98ff3da0e83724883477684b7154c9bfd1f
27. (PDF) Dataflow Process Networks - ResearchGate, accessed April 4, 2026, https://www.researchgate.net/publication/260685566_Dataflow_Process_Networks
28. Dataflow process networks - Proceedings of the IEEE - UCSB, accessed April 4,

2026, https://bears.ece.ucsb.edu/class/ece253/papers/lee_parks_ieee95.pdf
29. Deterministic Concurrency - Ptolemy Project, accessed April 4, 2026, https://ptolemy.berkeley.edu/~eal/books/DC/DeterministicConcurrency_Digital_01.pdf
30. Programming Z3 - Stanford CS Theory, accessed April 4, 2026, https://theory.stanford.edu/~nikolaj/programmingz3.html
31. cvc5: A Versatile and Industrial-Strength SMT Solver - Stanford Computer Science, accessed April 4, 2026, https://www-cs.stanford.edu/~preiner/publications/2022/BarbosaBBKLMMMN-TACAS22.pdf
32. SMT-based Model Checking of Transition Systems, accessed April 4, 2026, https://www.di.ens.fr/~pouzet/cours/mpri/cours4/cours-smt.pdf
33. Principles of Dependent Type Theory - Daniel Gratzer, accessed April 4, 2026, https://www.danielgratzer.com/papers/type-theory-book.pdf
34. The LLMbda Calculus: AI Agents, Conversations, and Information Flow - ResearchGate, accessed April 4, 2026, https://www.researchgate.net/publication/401132900_The_LLMbda_Calculus_AI_Agents_Conversations_and_Information_Flow
35. Securing AI Agents with Information-Flow Control - Microsoft Research, accessed April 4, 2026, https://www.microsoft.com/en-us/research/publication/securing-ai-agents-with-information-flow-control/
36. Language-Based Information-Flow Security - Cornell: Computer Science, accessed April 4, 2026, https://www.cs.cornell.edu/andru/papers/jsac/sm-jsac03.pdf
37. arXiv:2409.19091v2 [cs.CR] 10 Oct 2024, accessed April 4, 2026, https://arxiv.org/pdf/2409.19091